

CHAPTER - I

FUNCTION

1.1 :::: Introduction:

- i/. The most important criteria in writing and evaluating an algorithm is **time** to complete a task.
- ii/. The **computation time** is independent of programming languages, compiler and computer.

Subroutine:

Subroutine is the basic building block of computer programs. Subroutine is a small section of code which is used to perform a specific task.

Uses of Subroutine:

If bulk statements to be executed repeatedly then Subroutine can be used to finish that task. In Programming languages, Subroutine is also called as Function.

1.2 :::: Function:

Function is a unit of code. Function can be defined within a greater code structure. Function works on many kind of inputs, variants, expression and produce a concrete output.

Function Specification:

`a:=(24);`

In the given expression,

- i/. 'a' is a variable name.
- ii/. (24) is not an expression.
- iii/. (24) is a function definition to used to bind values to variable name 'a'.

Function Definition:

- i/. Function Definitions are syntactic block.

- ii). Function definitions have expressions.
- iii). All function definition are static.
- iv). There is no dynamic functions.

Example (in C++):

```
void add(int a, int b)
{
    int c;
    c=a+b;
    cout<<"c="<<c;
}
```

In the above example,

- a and b are parameters
- int is a data type of the parameter a and b
- Line no 3 to 5 are the body of the function

Parameters and Arguments:

Parameters are the variables in a function definition. Arguments are the values which passed to a function definition.

Example:

(requires : $x > 0$)

(returns : Square of x)

let square x

return : $x * x$

In the above example,

- i). 'let' is keyword to define a function
- ii). 'Square' is a name of the function

- iii). 'x' is a parameter.
- iv). The value passed to 'x' is called as argument.

Parameter without Type:

Parameters are the variables in a function definition. Arguments are the values which passed to a function definition.

Example:

(requires : $x > 0$)

(returns : Square of x)

let square x

if $x > 0$ then return : $x * x$

In the above example,

- i). 'let' is keyword to define a function
- ii). 'Square' is a name of the function
- iii). 'x' is a parameter.
- iv). The value passed to 'x' is called as argument.
- v). (Requires) is a precondition and (Returns) is a post condition.
- vi). Data type of x is not given.
- vii). The data type of 'x' is taken as int because 'x' is compared to 0 using equality operator.

Parameter with Type:

Parameters are the variables in a function definition. Arguments are the values which passed to a function definition.

Example:

(requires : $x > 0$)

(returns : Square of x)

```
let square (int x)
```

```
    if x>0 then return : x * x
```

In the above example,

- i). 'let' is keyword to define a function
- ii). 'Square' is a name of the function
- iii). 'x' is a parameter.
- iv). The value passed to 'x' is called as argument.
- v). (Requires) is a precondition and (Returns) is a post condition.
- vi). The data type of x is int which is explicitly.

Recursive Function:

A function definition which calls itself is called as recursive function.

Example:

```
let rec fact x
```

```
    if x=0 then
```

```
        return 1
```

```
    else
```

```
        fact=fact*fact(x-1)
```

In the above example,

- i). 'let rec' is keyword to define a recursive function
- ii). 'fact' is a name of the function
- iii). 'x' is a parameter

1.2 :::: Interface Vs Implementation:

Interface:

- i). Interface is a set of actions which can be executed by an object.
- ii). In OOPS, Interface is a description of all functions.
- iii). Interface just defines what an object can do.
- iv). Interface can't do anything.
- v). In Oops, classes are the interface.
- vi). Interface defines object's visibility.
- vii). Example : If you press light switch then light will on. You may not care about how light will on. Here, Switch is an interface.

Implementation:

- i). **Implementation** executes the instructions which are defined in an interface.
- ii). A class is a combination of Interface(data) with an implementation of that interface (function).
- iii). Object is an instance created from the class.
- iv). A class variable is called as Object.
- v). In Oops, how object is executed is an implementation.

Interface	Implementation
i). Interface just defines what an object can do.	Implementation executes the instructions which are defined in an interface.
ii). In Oops, classes are the interface.	In Oops, how object is executed is an implementation.

1.3.1 :::: Characteristics of Interface:

- i). Class template specifies an interface which enables an object to be created and executed properly.

- ii). Object's attributes(data) and behavior(functions) are controlled by sending functions to the object.

1.4 :::: Pure Functions:

- i). Pure functions are functions which will give exact result when same arguments are passed.
- ii). The evaluation of pure function doesn't cause any side effects to its output.
- iii). The pure function reads external memory but which doesn't change the output.
- iv). Example :

(a) `Sin(0)` always gives the value 0.

(b) `let square x`

`return : x*x`

gives the exact result for the same input.

(c) `Strlen(s)` gives the exact result for the same input.

a). Advantages of Pure Function:

(a) If a pure function is called several times with same arguments then the compiler actually call the function once.

(b) Example :

- `Strlen(s)` is a pure function.
- The compiler can remove the extra calls to `strlen(s)` for the same input.
- The compiler can execute the loop only once.

Why strlen is called as pure function?

`strlen(s)` is called as pure function because,

- i). `Strlen(s)` takes one variable as parameter and find its length.
- ii). `Strlen(s)` reads external memory but which doesn't change its output.
- iii). The evaluation of `strlen(s)` doesn't cause any side effects to its output
- iv). `Strlen(s)` functions which will give exact result when same arguments are passed.
- v). Example : `Strlen("Jaycees")` always gives the value 7.

ii). Therefore strlen(s) is called as pure function.

1.4.1 :::: Impure Functions:

- i). If an evaluation of function causes side effects then such function is called as Impure function.
- ii). Impure function may or may not return the same output for the same input.

1.4.2 :::: Side effects (Impure Functions):

- Impure function output is depend on the variables or function outside of its definition block.

i) Impure function output is depend on the variables

```
let random_number
    a:=random()
    if a>10 then
        return : a
    else
        return : 10
```

- In the above example, random() function returns different output when function is called.
- Therefore random_number function is called as Impure function.

ii) Impure function output is depend on function outside of its definition block

```
let y:=0
int inc int (x)
y:=y+x
return (y)
```

- In the above example, the value y is changed when each time inc() is called.
- Each time inc() gives different y value.
- Therefore inc() function is called as Impure function.

Pure Function	Impure Function
i). Pure function output is fully depends on its passed arguments.	Pure function output is not fully depends on its passed arguments.
ii). Pure functions which will give exact result when same arguments are passed.	Pure functions which will not give exact result when same arguments are passed.
oi). Pure function doesn't cause any side effects.	Pure function cause side effects.
iii). Pure function doesn't change the value of its arguments passed to it.	Pure function may change the value of its arguments passed to it..
io). Example : Strlen(s)	Example : random()

GCD Algorithm:

let rec gcd a,b

if $b \neq 0$ then

gcd b (a mod b)

else

return a;

Output:

gcd 13 27; returns 1

13	27	13
27	13	1
13	1	0
1	0	

GCD for 13 and 27 is 1

gcd 20536 7826; returns 2

20536	7826	4884
7826	4884	2942
4884	2942	1942
2942	1942	1000
1942	1000	942

1000	942	58
942	58	14
58	14	2
14	2	0
2	0	

GCD for 20536 and 7826 is 2

1.4.3 :::: Chameleons of Chromeland problem using Function:

-- inputs	:	a = A, b = B, c = C, a = b
-- outputs	:	a = b = 0, c = A+B+C

monochromatize (a, b, c)

-- inputs : a = A, b = B, c = C, a = b

-- outputs : a = b = 0, c = A+B+C

while a > 0

a, b, c := a-1, b-1, c+2

let rec monochromatize a b c :=

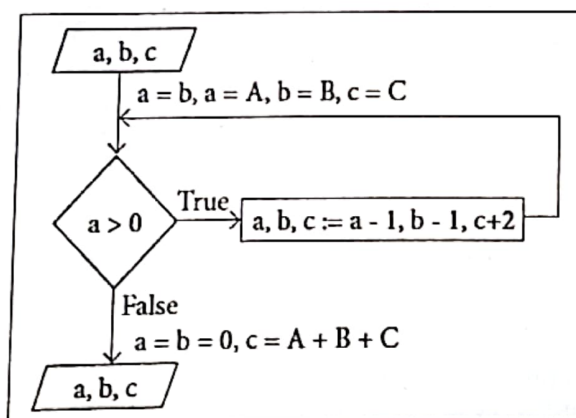
if a > 0 then

a, b, c := a-1, b-1, c+2

else

a := 0, b := 0, c := a + b + c

return c



iteration	a	b	c
0	4	4	4
1	3	3	8
2	2	2	10
3	1	1	12
4	0	0	14

Points to remember:

- Algorithms are expressed using statements of a programming language
- Subroutines are small sections of code that are used to perform a particular task that can be used repeatedly
- A function is a unit of code that is often defined within a greater code structure
- A function contains a set of code that works on many kinds of inputs and produces a concrete output
- Definitions are distinct syntactic blocks
- Parameters are the variables in a function definition and arguments are the values which are passed to a function definition through the function definition.
- When you write the type annotations the parentheses are mandatory in the function definition
- An interface is a set of action that an object can do
- Interface just defines what an object can do, but won't actually do it
- Implementation carries out the instructions defined in the interface
- Pure functions are functions which will give exact result when the same arguments are passed
- The variables used inside the function may cause side effects though the functions which are not passed with any arguments. In such cases the function is called impure function

WhatsApp Number :::: 9788852707

- Ask your doubt through my Whatsapp no
- Your doubt will be cleared in subsequent videos
- Don't Send any thing which is not relevant subject Like Good Morning, Hi...

1. Write algorithmic function definition to find the minimum among 3 numbers.
2. Write algorithmic recursive function definition to find the sum of n natural numbers.



Evaluation

Choose the best answer

Part - I

(1 Mark)

1. The small sections of code that are used to perform a particular task is called
(A) Subroutines (B) Files (C) Pseudo code (D) Modules
2. Which of the following is a unit of code that is often defined within a greater code structure?
(A) Subroutines (B) Function (C) Files (D) Modules
3. Which of the following is a distinct syntactic block?
(A) Subroutines (B) Function (C) Definition (D) Modules
4. The variables in a function definition are called as
(A) Subroutines (B) Function (C) Definition (D) Parameters
5. The values which are passed to a function definition are called
(A) Arguments (B) Subroutines (C) Function (D) Definition
6. Which of the following are mandatory to write the type annotations in the function definition?
(A) {} (B) () (C) [] (D) <>
7. Which of the following defines what an object can do?
(A) Operating System (B) Compiler (C) Interface (D) Interpreter
8. Which of the following carries out the instructions defined in the interface?
(A) Operating System (B) Compiler (C) Implementation (D) Interpreter
9. The functions which will give exact result when same arguments are passed are called
(A) Impure functions (B) Partial Functions
(C) Dynamic Functions (D) Pure functions

10. The functions which cause side effects to the arguments passed are called

(A) impure function

(B) Partial Functions

(C) Dynamic Functions

(D) Pure functions

Part - II

Answer the following questions

(2 Marks)

1. What is a subroutine?
2. Define Function with respect to Programming language.
3. Write the inference you get from $X := (78)$.
4. Differentiate interface and implementation.
5. Which of the following is a normal function definition and which is recursive function definition

i) let sum x y:

return x + y

ii) let disp :

print 'welcome'

iii) let rec sum num:

if (num!=0) then return num + sum (num-1)

else

return num

normal

recursive

Part - III

Answer the following questions

(3 Marks)

1. Mention the characteristics of Interface.
2. Why strlen is called pure function?
3. What is the side effect of impure function. Give example.
4. Differentiate pure and impure function.

Part - IV

(5 Marks)

Answer the following questions

1. What are called Parameters and write a note on
(i) Parameter without Type (ii) Parameter with Type
2. Identify in the following program

```
let rec gcd a b :=  
    if b <> 0 then gcd b (a mod b) else return a
```

- i) Name of the function *gcd*
 - ii) Identify the statement which tells it is a recursive function *base condition*
 - iii) Name of the argument variable *a, b*
 - iv) Statement which invoke the function recursively *gcd b (a mod b)*
 - v) Statement which terminates the recursion *return a*
3. Explain with example Pure and impure functions.
 4. Explain with an example interface and implementation.

REFERENCES

1. *Data Structures and Algorithms in Python* By Michael T. Goodrich, Roberto Tamassia and Michael H. Goldwasser.
2. *Data Structure and Algorithmic Thinking in Python* By Narasimha Karumanchi
3. <https://www.python.org>